1.- B. La clase StringBuilder en Java utiliza el método append() para agregar o concatenar cadenas de texto u otros tipos de datos al final de su secuencia interna.
2.- A. En la clase String de Java, el método replace(char oldChar, char newChar) se utiliza para reemplazar todas las apariciones de un carácter específico por otro, devolviendo una nueva cadena resultante (en este caso, "benene").
3.- A. Compara el contenido o valor real de las cadenas de texto. Como ambos objetos contienen la palabra "Bob", devuelve true.
4.- Longitud de la cadena: La cadena "Hello World" tiene exactamente 11 caracteres. Índices en Java: En Java, las cadenas (al igual que los arreglos) se indexan comenzando desde 0. Por lo tanto, los índices válidos para "Hello World" van desde 0 hasta 10.
5.- D. **Inmutabilidad de los objetos `String`**: En Java, las cadenas de texto son inmutables. Métodos como `toUpperCase()` **no modifican** el objeto existente en memoria, sino que crean y devuelven un *nuevo* objeto `String` con el resultado en mayúsculas (`"MESSAGE"`).
Falta de reasignación: Dentro del método `op2`, se llama a `s.toUpperCase();`, pero el valor devuelto no se asigna a ninguna variable (ni a `s` ni se devuelve para actualizar `msg`). Por lo tanto, el resultado de esa operación se ignora y se pierde.
Paso por valor: Java pasa las referencias de los objetos por valor. Aunque `s` apunta al mismo objeto que `msg`, cualquier cambio estructural o llamada a métodos que no reasigne la referencia no afecta a la variable original fuera del método.
Impresión final: Cuando se ejecuta `System.out.println(msg);` en el método `op3`, la variable `msg` sigue apuntando al objeto original con el texto en minúsculas, por lo que imprime **`message`**.
6.- A. En Java, las cadenas (String) son inmutables. Métodos como a.substring(0, 2); y a.concat("toc"); devuelven una nueva cadena con el resultado, pero no modifican ni reasignan el valor original de la variable a. Por lo tanto, a sigue siendo exactamente "tic".
7.- C. **str3.equals(str4)** → **true**: El método .equals() compara el **contenido o valor** de las cadenas. Como ambas cadenas almacenan exactamente el texto "Hello World", el resultado es verdadero.
str4.equals(str5) → **true**: De nuevo, compara el contenido de ambas variables y ambas contienen "Hello World", por lo que da verdadero.
str3 == str4 → **false**: El operador == compara **referencias de memoria** (si apuntan al mismo objeto exacto). Como str3 y str4 son objetos creados en distintas ubicaciones de memoria, la referencia es diferente.
str4 == str5 → **false**: Del mismo modo, str4 y str5 son dos instancias de objetos String independientes en el *heap*, por lo que sus referencias en memoria no coinciden.
8.- A. **str1 == str2**: Compara el objeto del *heap* (str1) con el objeto del *String Pool* (str2). Son objetos diferentes → **false**.
str2 == str3: Compara la referencia del *String Pool* (str2) con el resultado de .intern() (str3). Ambas apuntan al mismo objeto en el pool → **true**.
str1 == str3: Compara el objeto del *heap* (str1) con el objeto del *String Pool* (str3). Son diferentes → **false**.
9.- D. **s1 == s3** → **true**: Ambas variables apuntan al mismo literal en el String Pool.
s1 == s2 → **false**: Compara la referencia del String Pool (s1) con la referencia del objeto creado en el *heap* (s2). Son objetos diferentes.

s1 == s4 → **true**: s1 apunta al String Pool y s4 (s2.intern()) también devuelve la referencia ubicada en el String Pool, por lo que ambas apuntan al mismo lugar exacto en memoria.
10.- A. El método split("\\D"): Al dividir la cadena usando \D como separador, Java descarta las letras y extrae las partes numéricas que quedan entre ellas.El resultado es un arreglo con las cadenas: ["", "1", "2", "3", "4", "5", "6"] (la primera posición es una cadena vacía debido a que la cadena original comienza directamente con una letra).
11.- C. x = i + y;: Aquí ocurre el problema. Para sumar i (que es Integer) con y (que es int), Java intenta realizar un unboxing automático llamando internamente al método i.intValue(). Como i es null, intentar invocar un método sobre una referencia nula lanza inmediatamente una excepción de tipo NullPointerException en tiempo de ejecución.
12.- C. Prueba de escritorio